UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS

Curso: Programación II

Catedrático: Ing. Alejandro Díaz

MANUAL TÉCNICO

SIGELAB

Sistema de Gestión y Control de Laboratorios Universitarios

Grupo No. 01

Integrantes

Edgar Romario Chajon Chavez 3590-183531
Geovanni Orlando Vásquez Cotto 3590-24-657
Cristian Alexander Coromac López 3590-25-25489
Cristopher Stanley Díaz López 3590-25-20082
Kevin Armando Peñate Ambrocio 3590-25-20554
Fernanda Elizabeth López Hernandez 3590-25-19152
Manuel Estuardo Siney Rompich 3590-22-4807

Guatemala, 31 de octubre de 2026

ÍNDICE GENERAL

1. INTRODUCCIÓN Y OBJETIVO DEL DOCUMENTO....3
2. RESUMEN DEL PROYECTO EN NÚMEROS....3
3. DESCRIPCIÓN GENERAL DEL SISTEMA Y ROLES DE USUARIO....3
4. TECNOLOGÍAS UTILIZADAS Y POR QUÉ LAS ELEGIMOS....4
5. ARQUITECTURA DEL PROYECTO POR CAPAS....6
Organización de paquetes en src/main/java/umg/sigelab/....6
Estructura de carpetas del repositorio....7
6. MODELO DE BASE DE DATOS Y DIAGRAMA ENTIDAD-RELACIÓN....7
6.1 Diagrama Entidad-Relación (DER)....7
6.2 Diccionario resumido de las 16 tablas....7
6.3 Decisiones importantes en la base de datos....9
7. CICLO DE VIDA DE LOS PRÉSTAMOS Y DE LOS EQUIPOS....9
7.1 Estados de un préstamo....9
7.2 Estados de un equipo físico....9
8. CUMPLIMIENTO DE LAS 20 REGLAS DE NEGOCIO (RN-01 A RN-20)....10
9. MANEJO DE TRANSACCIONES Y CONTROL DE ERRORES....12
10. SEGURIDAD, CONTRASEÑAS Y CONTROL DE PERMISOS....13
11. FUNCIONAMIENTO DEL RECONOCIMIENTO FACIAL Y LECTOR QR....13
11.1 Flujo del inicio de sesión con segundo factor facial (RN-02)....13
11.2 Generación y lectura de códigos QR (RN-19)....14
12. CATÁLOGO DE PANTALLAS E INTERFAZ GRÁFICA (TEMA CELESTE Y BLANCO)....14
13. GUÍA DE INSTALACIÓN Y EJECUCIÓN PASO A PASO....15
Paso 1: Requisitos previos....15
Paso 2: Clonar el repositorio....16
Paso 3: Preparar la base de datos en SQL Server....16
Paso 4: Configurar la conexión en Java (db.properties)....16
Paso 5: Configurar e instalar el servicio facial (facial-service)....16
Paso 6: Ejecutar la aplicación....16
14. MATRIZ DE CASOS DE PRUEBA (CP-01 A CP-18)....17
15. TRABAJO EN EQUIPO Y CONTROL DE VERSIONES EN GIT....19
16. CONCLUSIONES....20

1. INTRODUCCIÓN Y OBJETIVO DEL DOCUMENTO

Este manual técnico explica cómo está construido por dentro el sistema **SIGELAB**, qué herramientas usamos para programarlo, cómo se organiza su base de datos en SQL Server y cómo se conectan las pantallas en Java Swing con el servicio de reconocimiento facial y lectura de códigos QR en Python.

El objetivo principal del documento es que cualquier compañero de clase, el catedrático o alguien que revise nuestro repositorio pueda entender cómo funciona el código, cómo instalar el proyecto paso a paso en su computadora y dónde están programadas las reglas de negocio que pide el curso.

2. RESUMEN DEL PROYECTO EN NÚMEROS

Para dar una idea rápida del tamaño del proyecto que armamos entre todos los integrantes, estos son los datos principales del repositorio:

Elemento del proyecto	Cantidad	Ubicación en el proyecto
Tablas en la base de datos	16 tablas	Carpeta `database/` (`SIGELAB_estructura_completa.sql` y migraciones)
Vistas SQL de apoyo	5 vistas	`vw_stock_laboratorio`, `vw_prestamos_resumen`, `vw_cola_despacho`, `vw_historial_equipo`, `vw_historial_mantenimiento`
Líneas de código en Java	~14,500 líneas	`src/main/java/umg/sigelab/`
Clases en `model` y `enums`	13 modelos y 14 enums	Paquetes `umg.sigelab.model` y `umg.sigelab.enums`
Ventanas y paneles gráficos	14 pantallas	Paquete `umg.sigelab.view` y ventanas de cámara en `facial-service`
Consultas SQL con `PreparedStatement`	94 consultas	Paquete `umg.sigelab.repository` (no usamos `Statement` normal en ningún lado)
Tipos de eventos de auditoría	14 acciones	Enum `AccionAuditoria` y tabla `Auditoria`
Endpoints del servicio en Python	6 rutas HTTP	`facial-service/app/main.py`
Casos de prueba documentados	18 casos (CP-01 a CP-18)	Sección 16 de este manual y pruebas con `pytest` en `facial-service/tests`

3. DESCRIPCIÓN GENERAL DEL SISTEMA Y ROLES DE USUARIO

SIGELAB es una aplicación de escritorio pensada para llevar el control de los equipos de cómputo y electrónica de los laboratorios de la universidad. En lugar de anotar préstamos

en papel o en hojas de Excel, el sistema controla todo el ciclo de vida de cada equipo físico: **Disponible**, **Prestado**, **Devuelto**, **Dañado**, **En Mantenimiento** y **De Baja**.

Además, implementamos un flujo ordenado de préstamos: los docentes y estudiantes pueden hacer su solicitud desde su cuenta eligiendo la sede y los equipos que necesitan, y luego el encargado de ese laboratorio revisa la orden, asigna las fechas en el calendario y confirma la entrega física (o hace préstamos directos en mostrador leyendo el código QR del equipo).

El sistema maneja **4 roles de usuario**, y dependiendo del rol con el que uno inicie sesión, la clase `Navegador.entrar()` abre la pantalla correspondiente:

Rol	Pantalla principal al entrar	Qué permisos tiene en el sistema
ADMIN	`MenuPrincipalForm`	Tiene acceso completo: administra usuarios, laboratorios/sedes, categorías de equipo, catálogo de productos, unidades físicas de equipos, reportes, auditoría y liberación de bloqueos. Puede operar en cualquier sede.
ENCARGADO	`PanelOperacionesForm`	Atiende únicamente el laboratorio que tiene asignado. Desde su barra lateral maneja órdenes pendientes de despacho, inventario de su sede, envíos y cierres de mantenimiento, préstamos en mostrador con QR y recepción de devoluciones.
DOCENTE	`SolicitudPrestamoForm`	Puede elegir una sede, ver qué modelos de equipos hay disponibles por categoría, armar su carrito de préstamo (1 a 30 días) y revisar su historial de préstamos.
ESTUDIANTE	`SolicitudPrestamoForm`	Tiene las mismas opciones de solicitud e historial que el docente. También es el rol que se asigna automáticamente cuando un alumno crea su cuenta desde la pantalla de registro.

4. TECNOLOGÍAS UTILIZADAS Y POR QUÉ LAS ELEGIMOS

Las versiones exactas están configuradas en el archivo `pom.xml` (para la parte de Java) y en `facial-service/requirements.txt` (para el servicio en Python):

Parte del sistema	Herramienta / Librería	Versión	Para qué la usamos
Lenguaje principal	Java (JDK)	21	Programación orientada a objetos de toda la lógica y pantallas.
Interfaz gráfica	Java Swing	Incluido en JDK 21	Construcción de las ventanas, tablas, diálogos y paneles con paleta Celeste y Blanco.
Gestor de dependencias	Apache Maven	3.13.0 (`compiler-plugin`)	Descargar las librerías automáticamente y compilar el proyecto.
Base de datos	Microsoft SQL Server	2022 / Azure SQL	Guardar toda la información de usuarios, inventario, préstamos y auditoría.
Conector de base de datos	`mssql-jdbc`	13.4.0.jre11	Conectar Java con SQL Server por medio de JDBC.
Encriptación de claves	`jBCrypt`	0.4	Guardar las contraseñas con hash y sal para que no se vean en texto plano en la base.
Calendarios gráficos	`JCalendar` (`com.toedter`)	1.4	Mostrar selectores de fecha (`JDateChooser`) al despachar préstamos y filtrar reportes.
Códigos QR en Java	Google `ZXing` (`core` y `javase`)	3.5.3	Generar las imágenes PNG de los códigos QR de cada equipo y leer QR desde archivo.
Servicio biométrico y QR	Python + FastAPI + Uvicorn	Python 3.12 / FastAPI 0.115.6	Levantar un servidor local en `127.0.0.1:8765` que controla la cámara web.
Visión por computadora	OpenCV + InsightFace + ONNX Runtime	OpenCV 4.10 / InsightFace 0.7.3	Abrir la cámara, detectar el rostro con el modelo `buffalo_l` y escanear códigos QR en vivo.
Seguridad biométrica	`cryptography` (Fernet) + `pyodbc`	44.0.0 / 5.2.0	Cifrar los vectores faciales (embeddings) antes de guardarlos en SQL Server.

¿Por qué elegimos estas tecnologías? Usamos **Java Swing con JDBC** porque es la base del curso de Programación II: escribir nosotros mismos las consultas SQL y manejar el `commit` y `rollback` de las transacciones nos ayudó a entender cómo viajan los datos desde la pantalla hasta la base. Para el reconocimiento facial decidimos apoyarnos en un microservicio local en **Python con InsightFace**, porque en Python las librerías de cámara e inteligencia artificial ya traen modelos listos que corren rápido en procesador normal (CPU) sin necesitar tarjeta de video dedicada.

[Figura pendiente de insertar: 4. Tecnologías utilizadas y por qué las elegimos]

5. ARQUITECTURA DEL PROYECTO POR CAPAS

Para no revolver el código de las pantallas con las consultas SQL, organizamos el proyecto en **capas**. La regla que seguimos en todo el proyecto es que una capa solo llama a la capa que tiene abajo:

```
View (Pantallas) → Controller → Service (Reglas de negocio) → Repository (SQL) → JDBC → SQL Server
```

[Figura pendiente de insertar: 5. Arquitectura del proyecto por capas]

Organización de paquetes en `src/main/java/umg/sigelab/`

Paquete	Qué archivos contiene	Ejemplo concreto
`config`	Configuración y lectura de credenciales para conectarse a SQL Server.	`DatabaseConnection.java`
`model`	Clases que representan las tablas o filas de resultados.	`Usuario`, `Equipo`, `Producto`, `Laboratorio`, `PrestamoResumen`
`enums`	Listas fijas de estados y roles para no equivocarnos al escribir textos.	`RolUsuario`, `EstadoEquipo`, `EstadoPrestamo`, `AccionAuditoria`
`view`	Todas las pantallas de la aplicación estilizadas con la paleta Celeste y Blanco.	`LoginForm`, `MenuPrincipalForm`, `PanelOperacionesForm`, `SolicitudPrestamoForm`, `ReportesForm`
`controller`	Clases puente entre las pantallas y los servicios.	`AuthController`, `PrestamoController`, `InventarioController`, `ConsultaController`
`service`	El corazón del sistema: validaciones, reglas de negocio y transacciones.	`AuthService`, `PrestamoService`, `DevolucionService`, `InventarioService`, `Permisos`
`repository`	Todas las consultas SQL parametrizadas con `PreparedStatement`.	`PrestamoRepository`, `InventarioRepository`, `ConsultaRepository`, `AuditoriaRepository`
`integration` y `qr`	Conexión HTTP con el servicio en Python y generación/lectura de códigos QR.	`FacialAuthClient`, `FacialServiceLauncher`, `QrCamaraClient`, `QrGenerator`, `QrLector`
`util`	Clases de ayuda para manejar transacciones, sesión activa y excepciones.	`Transaccion`, `Sesion`, `BusinessException`, `DataAccessException`,

Paquete	Qué archivos contiene	Ejemplo concreto
		`JdbcSupport`

Estructura de carpetas del repositorio

```
Proyecto-ProgramacionII/
├── pom.xml                        # Configuración de Maven y librerías de Java
├── README.md                      # Guía rápida del repositorio
├── build-sigelab.ps1              # Script para empaquetar el ejecutable SIGELAB.exe
├── database/                      # Scripts SQL de creación de tablas, vistas y ajustes
│   ├── SIGELAB_estructura_completa.sql
│   ├── SIGELAB_07_modulo_prestamos.sql
│   └── SIGELAB_08_hora_guatemala.sql
├── docs/                          # Documentación y manuales del equipo
├── facial-service/                # Microservicio en Python para rostro y lector QR
│   ├── app/                       # Código de FastAPI, cámara OpenCV, InsightFace y cifrado
│   ├── scripts/                   # Scripts de migración, diagnóstico (doctor.py) y lector QR
│   ├── sql/                       # Script de tablas faciales (PerfilFacial y
AuditoriaFacial)
│   ├── tests/                     # Pruebas unitarias en Python con pytest
│   ├── requirements.txt           # Librerías de Python
│   └── setup-facial.ps1           # Instalador automático del entorno virtual en Windows
└── src/main/
    ├── java/umg/sigelab/          # Código fuente en Java organizado por paquetes
    └── resources/
        └── db.properties.example  # Plantilla para configurar la conexión a SQL Server
```

6. MODELO DE BASE DE DATOS Y DIAGRAMA ENTIDAD-RELACIÓN

La base de datos en SQL Server está formada por **16 tablas** y **5 vistas**. Además de las tablas básicas de usuarios, equipos y préstamos, incluimos tablas para manejar el catálogo de modelos (`Producto`), el bloqueo automático de usuarios morosos (`BloqueoUsuario`), los vectores biométricos (`PerfilFacial`) y las secuencias automáticas de códigos.

6.1 Diagrama Entidad-Relación (DER)

[Figura pendiente de insertar: 6.1 Diagrama Entidad-Relación (DER)]

6.2 Diccionario resumido de las 16 tablas

N.º	Tabla	Para qué sirve	Restricciones e índices más importantes
1	`Rol`	Guarda los 4 roles fijos del sistema (`ADMIN`, `ENCARGADO`, `DOCENTE`, `ESTUDIANTE`).	`UNIQUE` en `nombre`; `CHECK` con los 4 valores permitidos.
2	`Laboratorio`	Representa cada sede o laboratorio físico donde están ubicados los equipos.	`UNIQUE` en `codigo` (RN-03); `CHECK (capacidad > 0)`.
3	`TipoEquipo`	Categorías generales de	`UNIQUE` en `codigo` y `nombre`;

N.º	Tabla	Para qué sirve	Restricciones e índices más importantes
		inventario (por ejemplo: Cómputo, Redes, Electrónica).	formato `CAT-XXXX-n`.
4	`SecuenciaTipoEquipo`	Lleva el correlativo para autogenerar el código de nuevas categorías.	Tabla de una sola fila (`CHECK (id_secuencia = 1)`).
5	`Usuario`	Datos de inicio de sesión, rol asignado, estado y laboratorio del encargado.	`UNIQUE` en `nombre_usuario` y `correo`; índice único para que solo haya un encargado activo por sede.
6	`PerfilFacial`	Guarda el vector matemático (embedding) cifrado del rostro de cada usuario.	`UNIQUE` en `id_usuario` (un perfil activo por persona).
7	`Producto`	Catálogo de modelos de equipos (marca, modelo, descripción y categoría).	`UNIQUE` en `codigo_producto`; FK hacia `TipoEquipo`.
8	`SecuenciaProducto`	Lleva el correlativo de códigos de producto según su categoría.	Llave primaria/foránea hacia `TipoEquipo`.
9	`Equipo`	Cada unidad física real con su código de inventario (QR), serie y estado actual.	`UNIQUE` en `codigo_inventario` (RN-04); `CHECK` con los 5 estados válidos.
10	`Prestamo`	Encabezado de cada solicitud o préstamo (solicitante, sede, fechas y estado).	`CHECK` de estados y validación de que las fechas de despacho y cierre sean coherentes.
11	`DetallePrestamo`	Lista de equipos específicos incluidos dentro de un préstamo.	`UNIQUE (id_prestamo, id_equipo)` (RN-06); llave foránea compuesta con `id_laboratorio`.
12	`Devolucion`	Registra cuándo y en qué condición (`BUENO` o `DANADO`) regresó cada equipo.	`UNIQUE` en `id_detalle` (RN-11: impide devolver dos veces el mismo equipo).
13	`BloqueoUsuario`	Bloquea a un usuario si tiene devoluciones parciales pendientes, atrasos o daños.	Índice único filtrado por préstamo y motivo cuando el bloqueo está `ACTIVO`.
14	`Mantenimiento`	Bitácora técnica de reparaciones preventivas o correctivas de un equipo.	Índice único filtrado que impide abrir dos mantenimientos al mismo tiempo para un mismo equipo.
15	`Auditoria`	Bitácora general que guarda quién hizo cada operación importante y en qué fecha.	Guarda `id_usuario`, `id_laboratorio`, `accion`, `entidad` y `descripcion`.
16	`AuditoriaFacial`	Bitácora técnica del servicio en Python con el puntaje (`score`) de cada lectura facial.	`CHECK` de tipos de evento (`ENROLL`, `VERIFY`, `DELETE`); FK a `Usuario`.

6.3 Decisiones importantes en la base de datos

1. **Separar `Producto` (modelo) de `Equipo` (unidad física):** Por ejemplo, el producto es *"Multímetro Digital Fluke 117"* y los equipos son las 10 unidades físicas con códigos `INV-0001` al `INV-0010` y distinto número de serie. Así el stock disponible se calcula contando cuántos equipos están en estado `DISPONIBLE` mediante la vista `vw_stock_laboratorio`, sin riesgo de que un contador manual se descuadre.
2. **Evitar mezclar equipos de distintas sedes:** En `DetallePrestamo` incluimos también `id_laboratorio` con llaves foráneas compuestas hacia `Prestamo` y hacia `Equipo`. Así el mismo SQL Server garantiza que jamás se meta un equipo del Laboratorio 2 en un préstamo del Laboratorio 1.
3. **El estado `VENCIDO` se calcula en tiempo real:** En vez de depender de una tarea programada que cambie una columna a medianoche, la vista `vw_prestamos_resumen` y `ConsultaRepository` revisan si la `fecha_esperada_devolucion` ya pasó y si todavía hay filas en `DetallePrestamo` sin su registro en `Devolucion` (RN-17).

7. CICLO DE VIDA DE LOS PRÉSTAMOS Y DE LOS EQUIPOS

7.1 Estados de un préstamo

Un préstamo pasa por los siguientes estados desde que se solicita hasta que se cierra:

[Figura pendiente de insertar: 7.1 Estados de un préstamo]

- **Nota sobre préstamos vencidos:** Mientras un préstamo está en estado `DESPACHADO`, si la fecha actual supera la `fecha_esperada_devolucion` y aún tiene equipos sin devolver, el sistema lo muestra automáticamente como **VENCIDO** en las pantallas de devoluciones y reportes.

7.2 Estados de un equipo físico

Cada unidad en la tabla `Equipo` tiene siempre uno de estos 5 estados (`EstadoEquipo`):

[Figura pendiente de insertar: 7.2 Estados de un equipo físico]

- **Protección de estados:** Un equipo que está `PRESTADO` no se puede mandar a mantenimiento ni dar de baja manualmente desde el inventario; primero tiene que

registrarse su devolución. Tampoco se puede volver a activar un equipo que ya pasó a `BAJA`.

8. CUMPLIMIENTO DE LAS 20 REGLAS DE NEGOCIO (RN-01 A RN-20)

Todas las reglas pedidas en la guía del proyecto están programadas dentro de la capa `service` (para que no dependan solo de los botones de la pantalla) y además tienen respaldo en las restricciones de SQL Server:

Código	Descripción de la regla	Clase y método en Java donde se valida	Respaldo en SQL Server
RN-01	Un usuario `INACTIVO` no puede iniciar sesión.	`AuthService.autenticar()`	`CHECK` de estado en tabla `Usuario`
RN-02	La contraseña no basta; se requiere verificación facial exitosa.	`LoginForm` + `FacialAuthClient.verify()` + `AuthService`	Tabla `PerfilFacial` y `AuditoriaFacial`
RN-03	El código de cada laboratorio debe ser único.	`LaboratorioAdminService`	Restricción `UQ_Laboratorio_Codigo`
RN-04	El código de inventario de cada equipo debe ser único.	`EquipoService` / `InventarioService`	Restricción `UQ_Equipo_Codigo`
RN-05	Un préstamo debe incluir al menos un equipo.	`PrestamoService.solicitar()` y `crearYDespachar()`	Validación transaccional en creación
RN-06	No se puede repetir el mismo equipo dentro de un préstamo.	`PrestamoService.solicitar()` y `crearYDespachar()`	Restricción `UQ_Detalle_Prestamo_Equipo`
RN-07 / 08 / 14	Solo se pueden prestar equipos que estén en estado `DISPONIBLE`.	`PrestamoService` (`unidadLibre` y `cambiarEstadoEquipo`)	`CHECK` de estados en `Equipo`
RN-09	Al despachar un préstamo,	`PrestamoService.despachar()` y `crearYDespachar()`	Actualización dentro de la transacción

Código	Descripción de la regla	Clase y método en Java donde se valida	Respaldo en SQL Server
	todos sus equipos pasan a `PRESTADO`.		
RN-10	Solo se pueden devolver equipos pendientes de un préstamo `DESPACHADO`.	`DevolucionService.devolver()`	Llave foránea hacia `DetallePrestamo`
RN-11	Un equipo no se puede devolver dos veces en el mismo préstamo.	`DevolucionService.devolver()`	Restricción `UQ_Devolucion_Detalle`
RN-12	Si un equipo se devuelve `BUENO`, su estado vuelve a `DISPONIBLE`.	`DevolucionService.devolver()`	`CK_Devolucion_Condicion`
RN-13	Si se devuelve `DANADO`, pasa a `DANADO` y exige escribir la observación.	`DevolucionService.devolver()`	`CK_Devolucion_Condicion`
RN-15	Al cerrar un mantenimiento, el equipo pasa a `DISPONIBLE` o a `BAJA`.	`InventarioService.finalizarMantenimiento()`	Restricción `CK_Mant_Cierre`
RN-16	En una devolución parcial, el préstamo sigue activo (`DESPACHADO`).	`DevolucionService.devolver()` (`contarPendientes`)	Vista `vw_prestamos_resumen`
RN-17	Un préstamo con fecha vencida y equipos pendientes se marca como vencido.	`ConsultaRepository.contarVencidos()` y `vencidos()`	Columna calculada en `vw_prestamos_resumen`

Código	Descripción de la regla	Clase y método en Java donde se valida	Respaldo en SQL Server
RN-18	Todas las operaciones críticas dejan registro en la bitácora de auditoría.	`AuditoriaRepository.registrar()` en cada `Service`	Tabla `Auditoria`
RN-19	Escanear un código QR solo identifica al equipo; no altera su estado por sí solo.	`QrCampo`, `InventarioService.ficha()` y `buscarPorCodigo()`	Consultas de solo lectura (`SELECT`)
RN-20	Las validaciones viven en la capa de negocio (`Service`), no en la interfaz.	Clases del paquete `umg.sigelab.service` y `Permisos`	`CHECK`, `UNIQUE` y `FOREIGN KEY`

9. MANEJO DE TRANSACCIONES Y CONTROL DE ERRORES

Cuando una operación modifica varias tablas al mismo tiempo (por ejemplo, despachar un préstamo actualiza la tabla `Prestamo`, cambia el estado de varios registros en `Equipo` e inserta filas en `Auditoria`), no podemos usar el `autoCommit` normal de JDBC porque si fallara a la mitad quedarían datos incompletos.

Para resolverlo creamos la clase utilitaria `Transaccion.java`, que desactiva el `autoCommit`, ejecuta todos los pasos con la misma conexión `Connection` y hace `commit()` solo si todo salió bien, o `rollback()` automático si ocurre cualquier excepción:

```
public static <T> T ejecutar(String operacion, Trabajo<T> trabajo) {
   try (Connection cn = DatabaseConnection.getConnection()) {
       cn.setAutoCommit(false);
       try {
           T resultado = trabajo.ejecutar(cn);
           cn.commit();
           return resultado;
       } catch (SQLException | RuntimeException e) {
           cn.rollback();
           throw e;
       }
   } catch (SQLException e) {
       throw JdbcSupport.error(operacion, e);
   }
}
```

Además, para que al usuario nunca le aparezca un error técnico feo de SQL en pantalla, separamos las excepciones en dos clases propias:

- **`BusinessException`:** Se lanza cuando el usuario intenta hacer algo no permitido por las reglas de negocio (por ejemplo: *"Solo se pueden actualizar equipos DISPONIBLES"* o *"El equipo ya fue devuelto"*). La vista la atrapa y muestra un aviso claro.
- **`DataAccessException`:** Se lanza mediante `JdbcSupport.error(...)` cuando ocurre un problema de red o de base de datos, guardando el detalle técnico en consola y mostrando un mensaje amigable en la ventana.

10. SEGURIDAD, CONTRASEÑAS Y CONTROL DE PERMISOS

1. **Verificación de permisos en el `Service`:** La clase `Permisos.java` revisa en cada método si el usuario que tiene la sesión iniciada tiene permiso para ejecutar esa acción (`exigirAdmin`, `exigirOperador` validando que el encargado pertenezca a ese laboratorio, y `exigirSolicitante`).
2. **Hashes de contraseña con `jBCrypt`:** Nunca guardamos contraseñas en texto plano. Al crear un usuario o cambiar su clave se genera un hash con `BCrypt.hashpw(password, BCrypt.gensalt(12))`. En el login, incluso si el nombre de usuario no existe, `AuthService` ejecuta una comparación BCrypt contra un hash ficticio para que un atacante no pueda adivinar qué usuarios existen midiendo el tiempo de respuesta.
3. **Prevención de Inyección SQL:** Todas las 94 consultas del paquete `repository` usan `PreparedStatement` con parámetros `?`.
4. **Cifrado de vectores faciales:** En la tabla `PerfilFacial` no se guardan fotos de los alumnos ni vectores en claro; el servicio en Python cifra el arreglo matemático con `Fernet` (AES) usando la llave `FACIAL_ENCRYPTION_KEY` antes de hacer el `INSERT`.

11. FUNCIONAMIENTO DEL RECONOCIMIENTO FACIAL Y LECTOR QR

11.1 Flujo del inicio de sesión con segundo factor facial (RN-02)

[Figura pendiente de insertar: 11.1 Flujo del inicio de sesión con segundo factor facial (RN-02)]

- **Arranque automático del servicio:** Cuando arranca `Main.java`, la clase `FacialServiceLauncher` verifica si el entorno de Python ya está preparado (`.setup-ok`); si falta, ejecuta la instalación inicial y levanta el servidor FastAPI en segundo plano (`127.0.0.1:8765`). En la esquina del Login se muestra en vivo si el servicio está activo y al hacer clic sobre el indicador o el botón **Servicio facial...** se abre la ventana `ServicioFacialForm` para diagnosticarlo o reiniciarlo.
- **Captura guiada:** Durante el registro facial (`/face/enroll`) el sistema guía al usuario en 4 fases (mirar al frente, girar a la izquierda, girar a la derecha y mirar al frente) tomando 20 muestras de calidad antes de guardar su perfil.

11.2 Generación y lectura de códigos QR (RN-19)

- **Generación:** Al registrar un equipo físico en `EquipoForm`, la clase `QrGenerator` usa la librería `ZXing` para crear un archivo PNG en la carpeta `qr-equipos/` con el código de inventario (por ejemplo `INV-0001.png`) y permite visualizarlo o exportarlo desde la ventana.
- **Lectura por cámara, archivo o lector USB:** El componente reutilizable `QrCampo` está integrado en las pantallas de Inventario, Mantenimiento, Préstamo en Mostrador, Devoluciones y Reportes. Permite escribir el código con teclado/escáner USB, cargar una imagen PNG desde la computadora o presionar **"Leer QR (cámara)"**, que abre la ventana de cámara con un recuadro guía celeste para detectar el QR automáticamente.

12. CATÁLOGO DE PANTALLAS E INTERFAZ GRÁFICA (TEMA CELESTE Y BLANCO)

Todas las pantallas del sistema fueron diseñadas y unificadas bajo una misma **paleta de colores Celeste y Blanco** para mantener una apariencia limpia, moderna y fácil de leer:

- **Celeste Principal (`#2B8FD6`):** Barra lateral de navegación, encabezados de tablas y botones de acción principal.
- **Celeste Medio (`#63B3ED`):** Bordes de campos de texto, separadores y efecto hover en botones.
- **Celeste Suave (`#BEE3F8`):** Selección de filas en tablas, bordes de tarjetas y botones secundarios.
- **Blanco Hielo (`#F0F8FF`):** Fondo general de las ventanas.
- **Blanco Puro (`#FFFFFF`):** Tarjetas de formularios, paneles de filtros y cajas de texto.
- **Gris Oscuro (`#1A202C`):** Texto principal y etiquetas en fuente `Segoe UI`.

N.º	Ventana / Clase	Rol que la usa	Función principal
1	`LoginForm`	Todos	Inicio de sesión con usuario, contraseña, verificación facial y accesos a registro y recuperación.
2	`ServicioFacialForm`	Todos / Soporte	Panel de control para instalar, iniciar, detener y ver el estado del servicio facial en Python.
3	`MenuPrincipalForm`	`ADMIN`	Menú central del administrador con acceso a todos los catálogos, operaciones y reportes.
4	`UsuarioForm` y diálogos de cuenta	`ADMIN` / Público	Gestión de usuarios, auto-registro de estudiantes y recuperación de contraseña con rostro.
5	`LaboratorioForm`	`ADMIN`	Administración de sedes/laboratorios (código único, nombre, ubicación, capacidad y estado).
6	`TipoEquipoForm`	`ADMIN`	Gestión de categorías de

N.º	Ventana / Clase	Rol que la usa	Función principal
			equipos con generación automática de código (`CAT-XXXX-n`).
7	`ProductoForm`	`ADMIN`	Catálogo de modelos de equipos (código, nombre, categoría, marca, modelo, descripción y estado).
8	`EquipoForm`	`ADMIN`	Registro de unidades físicas por sede, número de serie y visor/exportador de código QR en PNG.
9	`PanelOperacionesForm`	`ENCARGADO` / `ADMIN`	Centro de operaciones con barra lateral celeste y 5 subpaneles: *Órdenes pendientes*, *Inventario*, *Mantenimiento*, *Crear préstamo (QR)* y *Devoluciones*.
10	`SolicitudPrestamoForm`	`DOCENTE` / `ESTUDIANTE`	Portal con barra lateral para armar solicitudes de préstamo por sede/categoría y ver su historial.
11	`ReportesForm`	`ADMIN` / `ENCARGADO`	Ventana con 8 pestañas: Dashboard con tarjetas de totales, equipos por estado, préstamos por fecha, vencidos, historial por equipo, mantenimientos, bloqueos y auditoría.
12	`PreviewWindow` (Registro Facial)	Todos	Ventana de cámara OpenCV con encabezado celeste que guía las 4 fases del registro biométrico.
13	`PreviewWindow` (Verificación Facial)	Todos	Ventana de cámara OpenCV que valida el rostro y la prueba de vida al iniciar sesión.
14	`qr_scan.py` (Lector QR por Cámara)	`ENCARGADO` / `ADMIN`	Ventana de cámara OpenCV con recuadro guía celeste para escanear el código QR de un equipo.

13. GUÍA DE INSTALACIÓN Y EJECUCIÓN PASO A PASO

Sigue estos pasos para instalar y correr el proyecto desde cero en cualquier computadora con Windows:

Paso 1: Requisitos previos

1. **Java JDK 21** (o superior compatible con `--release 21`) e **IntelliJ IDEA**.
2. **Microsoft SQL Server** (edición Express, Developer o Azure SQL) y **ODBC Driver 18 for SQL Server**.

3. **Python 3.11 o 3.12** instalado en el sistema.
4. **Cámara web** conectada o integrada en la laptop.

Paso 2: Clonar el repositorio

```
git clone https://github.com/RomarioChajon/Proyecto-ProgramacionII.git
cd Proyecto-ProgramacionII
```

Paso 3: Preparar la base de datos en SQL Server

Abrir SQL Server Management Studio (SSMS) o DBeaver y ejecutar los scripts de la carpeta `database/` en este orden:

1. `database/SIGELAB_estructura_completa.sql` (crea la base de datos `SIGELAB`, las 16 tablas, restricciones y vistas).
2. `database/SIGELAB_07_modulo_prestamos.sql` (asegura las columnas del flujo de solicitudes y modelos).
3. `database/SIGELAB_08_hora_guatemala.sql` (configura la zona horaria de Guatemala para los registros automáticos).

Paso 4: Configurar la conexión en Java (`db.properties`)

1. Ir a la carpeta `src/main/resources/`.
2. Copiar el archivo `db.properties.example` y renombrar la copia a `db.properties`.
3. Colocar los datos de tu servidor SQL Server:

```
db.url=jdbc:sqlserver://
localhost:1433;databaseName=SIGELAB;encrypt=true;trustServerCertificate=true;
db.user=sa
db.password=TuPasswordDeSqlServer
```

Paso 5: Configurar e instalar el servicio facial (`facial-service`)

1. Entrar a la carpeta `facial-service/`, copiar `.env.example` a `.env` y verificar que `DB_SERVER`, `DB_NAME`, `DB_USER` y `DB_PASSWORD` apunten a tu base de datos `SIGELAB`.
2. Abrir PowerShell en la carpeta `facial-service/` y ejecutar el instalador automático:

```
powershell -ExecutionPolicy Bypass -File .\setup-facial.ps1
```

(Este script crea el entorno virtual `.venv`, instala FastAPI, OpenCV, InsightFace y ONNX Runtime, ejecuta la migración de las tablas faciales, descarga el modelo `buffalo_l` y ejecuta `scripts.doctor` para confirmar que todo esté listo).

Paso 6: Ejecutar la aplicación

1. Abrir el proyecto en **IntelliJ IDEA**.
2. Ejecutar la clase principal `umg.sigelab.Main`.
3. Al abrirse la ventana de Login, el sistema iniciará automáticamente el servicio facial en segundo plano y mostrará el estado **"Servicio facial activo"**.

14. MATRIZ DE CASOS DE PRUEBA (CP-01 A CP-18)

A continuación se documentan los 18 casos de prueba obligatorios que validan el cumplimiento de las reglas de negocio del sistema:

Código	Caso de prueba	Pasos realizados	Resultado esperado	Clase que lo valida	Resultado
CP-01	Login con credenciales válidas y rostro correcto	Ingresar usuario activo, contraseña correcta y mirar al frente en la cámara.	Se valida el rostro (`matched=true`), se audita `LOGIN_EXITOSO` y abre el menú del rol.	`AuthService` y `FacialAuthClient`	**Aprobado**
CP-02	Login con credenciales válidas y rostro incorrecto	Ingresar usuario y clave válidos, pero que otra persona se coloque frente a la cámara.	El servicio devuelve `matched=false`, se audita `LOGIN_FALLIDO` y se niega el acceso (RN-02).	`LoginForm` y `AuthService`	**Aprobado**
CP-03	Registrar un equipo con código de inventario repetido	Intentar guardar un equipo nuevo escribiendo un código que ya existe (ej. `INV-0001`).	El sistema muestra advertencia indicando que el código ya existe y no duplica el registro (RN-04).	`EquipoService` y `UQ_Equipo_Codigo`	**Aprobado**
CP-04	Prestar un equipo en estado `DISPONIBLE`	Crear o despachar un préstamo con un equipo disponible.	El préstamo pasa a `DESPACHADO` y el equipo cambia a estado `PRESTADO` (RN-07, RN-09).	`PrestamoService.despachar()`	**Aprobado**
CP-05	Intentar prestar de nuevo un equipo que ya está `PRESTADO`	Escanear o seleccionar un equipo prestado para otro préstamo.	El sistema rechaza la operación indicando que solo se prestan equipos disponibles (RN-08).	`PrestamoService.crearYDespachar()`	**Aprobado**
CP-06	Intentar prestar un equipo `DANADO`	Escanear en mostrador el código de un equipo marcado como dañado.	Operación bloqueada; muestra la ficha del equipo e impide agregarlo al préstamo (RN-08).	`PrestamoService.crearYDespachar()`	**Aprobado**

Código	Caso de prueba	Pasos realizados	Resultado esperado	Clase que lo valida	Resultado
CP-07	Intentar prestar un equipo en `MANTENIMIENTO`	Escanear en mostrador un equipo que está en reparación.	Operación bloqueada; el sistema informa que está en mantenimiento (RN-08).	`PrestamoService.crearYDespachar()`	**Aprobado**
CP-08	Intentar prestar un equipo dado de `BAJA`	Escanear en mostrador el código de un equipo dado de baja.	Operación bloqueada; un equipo de baja no puede prestarse (RN-08).	`PrestamoService.crearYDespachar()`	**Aprobado**
CP-09	Escanear el QR de un equipo `DISPONIBLE`	Leer el código QR desde cámara o imagen PNG en `InventarioPanel` o `PrestamoMostradorPanel`.	Identifica el equipo sin cambiar su estado en la lectura y permite agregarlo al préstamo (RN-19).	`QrCampo` e `InventarioService.ficha()`	**Aprobado**
CP-10	Escanear el QR de un equipo `PRESTADO`	Leer el QR de un equipo prestado en la pantalla de préstamo en mostrador.	Muestra los datos actuales del equipo pero impide agregarlo a un nuevo préstamo (RN-19).	`PrestamoMostradorPanel`	**Aprobado**
CP-11	Devolver un equipo en condición `BUENO`	En `DevolucionesPanel`, seleccionar el equipo prestado, elegir `Buen estado` y confirmar.	Se inserta la devolución y el equipo vuelve inmediatamente a `DISPONIBLE` (RN-12).	`DevolucionService.devolver()`	**Aprobado**
CP-12	Devolver un equipo en condición `DANADO`	Seleccionar el equipo, elegir condición `Dañado`, escribir la observación del daño y confirmar.	Se exige la observación, se registra la devolución y el equipo pasa a estado `DANADO` (RN-13).	`DevolucionService.devolver()`	**Aprobado**
CP-13	Intentar devolver dos veces el mismo equipo	En un préstamo con devolución parcial, seleccionar un renglón que ya dice `Devuelto`.	El sistema advierte que ese equipo ya fue devuelto y bloquea el segundo registro (RN-11).	`DevolucionesPanel` y `DevolucionService`	**Aprobado**
CP-14	Devolución	En un préstamo	Los 2 equipos	`DevolucionSer`	**Aprobado**

Código	Caso de prueba	Pasos realizados	Resultado esperado	Clase que lo valida	Resultado
	parcial (devolver 2 de 3 equipos)	de 3 equipos, marcar solo 2 y presionar *Devolver seleccionados*.	cambian de estado, el préstamo continúa `DESPACHADO` con 1 pendiente y se crea bloqueo parcial (RN-16).	`vice.devolver()`	
CP-15	Enviar un equipo `DANADO` a mantenimiento	En `InventarioPanel`, elegir un equipo dañado, presionar *Mantenimiento* y llenar motivo y técnico.	Se abre el registro en la tabla `Mantenimiento` y el equipo pasa a `MANTENIMIENTO`.	`InventarioService.cambiarEstado()`	**Aprobado**
CP-16	Finalizar mantenimiento como `REPARADO`	En `MantenimientoPanel`, seleccionar el equipo y presionar *Finalizar: reparado*.	Se cierra la orden de mantenimiento con fecha de fin y el equipo vuelve a `DISPONIBLE` (RN-15).	`InventarioService.finalizarMantenimiento()`	**Aprobado**
CP-17	Finalizar mantenimiento como `NO_REPARABLE`	En `MantenimientoPanel`, seleccionar el equipo y presionar *Finalizar: no reparable*.	Se cierra el mantenimiento y el equipo pasa definitivamente al estado `BAJA` (RN-15).	`InventarioService.finalizarMantenimiento()`	**Aprobado**
CP-18	Detección de préstamo vencido en reportes	Consultar la pestaña *Préstamos vencidos* en `ReportesForm` cuando la fecha esperada ya pasó y hay equipos sin devolver.	El préstamo aparece listado como vencido sin necesidad de modificar manualmente su fila (RN-17).	`ConsultaRepository.vencidos()`	**Aprobado**

15. TRABAJO EN EQUIPO Y CONTROL DE VERSIONES EN GIT

El desarrollo se llevó a cabo utilizando **Git y GitHub** en el repositorio grupal (`RomarioChajon/Proyecto-ProgramacionII`), trabajando mediante ramas individuales por integrante (`integracion`, `geovanni-proyecto`, `manuel_rama`, `Kevin_Rama`, `cdiazl10_rama`, `ccoromacl_rama`, `elizabeth_Rama`) e integrando los avances de forma progresiva a lo largo del semestre:

1. **Base de datos y modelo relacional:** Diseño de las 16 tablas, vistas de inventario/préstamos y restricciones de integridad en SQL Server.
2. **Módulos de catálogos e inventario:** CRUD de usuarios, laboratorios, categorías de equipo, productos y unidades físicas con generación de códigos QR.
3. **Módulo transaccional de préstamos, devoluciones y mantenimiento:** Flujo de solicitudes para docentes y estudiantes, panel operativo por sede para encargados, control de bloqueos y bitácora de auditoría.
4. **Integración biométrica y diseño visual:** Microservicio de reconocimiento facial y lector QR con cámara en Python, empaquetado de la aplicación e implementación de la interfaz gráfica con la paleta Celeste y Blanco en las 14 ventanas del sistema.

16. CONCLUSIONES

1. Separar la aplicación en capas (`View`, `Controller`, `Service` y `Repository`) nos permitió programar y corregir las pantallas Swing con el tema Celeste y Blanco sin riesgo de romper las consultas SQL ni las validaciones de negocio.
2. El manejo manual de transacciones JDBC mediante una clase centralizada (`Transaccion.java`) garantizó que operaciones delicadas como el despacho de préstamos o las devoluciones parciales nunca dejen equipos en estados inconsistentes si ocurre algún error a mitad del proceso.
3. La combinación de Java Swing con un servicio local en Python para la cámara demostró ser una solución práctica para integrar reconocimiento facial (`InsightFace`) y lectura de códigos QR en tiempo real dentro de un proyecto académico de escritorio.